claude-windows / dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3\subagents\workflows\wf_f1431155-4a0\agent-ab6bf2648970a6e94.jsonl`
- SHA-256: `9f038968bbb8b3721e61d2fce2d6fec34e2b2907752d41a82eee6624f9cd728d`
- Source modified: `2026-06-21T07:28:38+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_f1431155-4a0`
- session_id: `dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3`

Transcript

1. user (2026-06-21T07:27:10.266Z)

```
You are reviewing milestone M1 of the COMPUTE_DECOUPLED_SERVING project in the repo at
C:/Users/avidu/Projects/badminton-highlight-indexer.
The spec is docs/COMPUTE_DECOUPLED_SERVING/README.md (read §2 D7-D15, §4.3 "Profile selection",
and §7 the tracker row M1). M1 must be: PURE, ADDITIVE, with deployment.profile='' (the default)
== today's behaviour, ZERO core change. A *deployment profile* is opt-in and applies a coherent
preset bundle of EXISTING knobs (indexing.detector_impl/skip_ai_handoff, storage.backend) plus
the
new deployment.compute_target. Precedence: built-in defaults < profile preset < config.json <
env.
D15: auto-detect SUGGESTS, never auto-applies (cloud spend never entered silently).

Files changed (read them all, plus the existing tests/test_config.py to understand the prior
contract):
backend/config/presets.py (NEW)
backend/config/models.py (added DeploymentConfig + AppConfig.deployment field)
backend/config/loader.py (added _deep_merge, _resolve_explicit_profile, preset application)
backend/config/__init__.py (re-exports)
tests/test_config_deployment.py (NEW)

Ground EVERY claim in the actual code with a file:line citation. Do NOT speculate. If you assert
a
bug, give the exact input that triggers it and the wrong-vs-right output. Rate confidence
honestly;
prefer FEWER, HIGH-CONFIDENCE findings. A clean bill ([] findings) is a valid and good result.

Four reviewers produced these raw findings (JSON):
[
  {
    "title": "Explicit empty-string compute_target override silently blanks an active profile's
compute_target without the mismatch warning",
    "severity": "minor",
    "confidence": "medium",
    "location": "backend/config/loader.py:156-163",
    "detail": "Trigger: config.json = {\"deployment\": {\"profile\": \"cloud-serving\",
\"compute_target\": \"\"}} (verified by running load_config). The cloud-serving preset sets
compute_target=\"cloud_run\"; _deep_merge overlays the file's compute_target=\"\" on top, so
merged compute_target becomes \"\". In the re-stamp block, ct =
merged[\"deployment\"].get(\"compute_target\") == \"\" (falsy), so the guard `if ct and
canonical and ct != canonical` short-circuits and NO warning fires. Result:
cfg.deployment.profile==\"cloud-serving\" but cfg.deployment.compute_target==\"\" ? an active
cloud profile with a blanked, inconsistent compute_target, recorded silently. This is asymmetric
with the documented/tested path where overriding to a WRONG NON-EMPTY target (e.g. cloud_run
under cpu-mock) DOES warn (test_config_json_can_override_compute_target_with_warning). Wrong:
```

```
silent blanking. Right: an active profile whose canonical compute_target is overridden to empty
should still be surfaced. IMPACT IS BOUNDED: compute_target is documented as inert in M1
('recorded for observability; consumed by the ComputeBackend seam from M2+', models.py:338-339),
so there is zero behavioural effect this milestone ? this is an observability/honesty gap, not a
correctness bug, and should be revisited before M2 wires compute_target to a real ComputeBackend
(where an empty compute_target under cloud-serving could select the wrong/no backend).",
    "fix": "In the re-stamp block, after applying overrides, treat an empty compute_target under
an active known profile as 'fall back to the canonical and/or warn'. E.g. change the guard so an
empty ct is backfilled to `canonical` (keeping the observability record honest), or warn
explicitly: `if canonical and ct != canonical: logger.warning(... empty-or-divergent ...)`. Add
a unit test pinning {profile: cloud-serving, compute_target: \"\"} ? compute_target resolves to
cloud_run (or warns), mirroring test_config_json_can_override_compute_target_with_warning.",
    "lens": "precedence"
  },
  {
    "title": "Invalid DEPLOYMENT_PROFILE env value silently suppresses a valid config.json
profile, leaving an incoherent (recorded-but-not-applied) config",
    "severity": "minor",
    "confidence": "high",
    "location": "backend/config/loader.py:47-59,136-147",
    "detail": "_resolve_explicit_profile (loader.py:51-53) returns the DEPLOYMENT_PROFILE env
value as soon as it is non-blank, WITHOUT validating it, and never falls through to the
config.json profile. So when the env has a typo AND config.json sets a valid profile, the bad
env value wins, is then rejected by is_known_profile (loader.py:138), profile is reset to \"\",
and the loader takes the no-profile else branch (loader.py:166) ? NO preset is applied. But
model_validate still runs on the shallow-merged dict that contains
file_data['deployment']['profile'], so the *recorded* value disagrees with the *applied*
behaviour. Triggering input: env DEPLOYMENT_PROFILE='banana' + config.json {'deployment':
{'profile': 'cpu-mock'}}. Observed (verified by running load_config): cfg.deployment.profile ==
'cpu-mock' but cfg.indexing.detector_impl == 'auto' (NOT the 'stub' the cpu-mock preset would
set), skip_ai_handoff/storage.backend also unconfigured. Right behaviour: a typo'd env override
should fall back to the valid config.json profile (apply the cpu-mock preset,
detector_impl='stub'), OR the recorded cfg.deployment.profile should be forced to '' so it
cannot claim an inactive profile is active. The loader.py:139-142 comment ('A bad value coming
only from the env override is ignored here') states the intended contract ? but the env value is
not merely ignored, it actively clobbers the file's valid profile. NOTE: this is NOT a backward-
compat / zero-core-change violation ? the default profile='' path is fully unaffected (verified:
repo config.json has no deployment section, loads via the else branch byte-identically, all 52
config tests + 80 config-adjacent tests pass). It is a robustness gap in the new opt-in
precedence logic, reachable only under a specific misconfiguration (env typo co-existing with a
valid file profile), which is why it is minor rather than a blocker.",
    "fix": "Validate the env-supplied profile inside _resolve_explicit_profile (or right after
reading it): if os.environ['DEPLOYMENT_PROFILE'] is set but not is_known_profile(it), warn and
fall through to the config.json deployment.profile instead of returning the bad value.
Alternatively, in the else branch, scrub merged['deployment']['profile'] to '' whenever no
preset was applied so the recorded profile can never assert an inactive state. Add a unit test:
env='banana' + file profile='cpu-mock' must yield either (a) cpu-mock fully applied or (b)
profile=='' with auto detector_impl ? never the current split state.",
    "lens": "backcompat"
  },
  {
    "title": "Torch-free guarantee is asserted only in docstrings, never tested ? a regression
to eager torch import would pass every test",
    "severity": "major",
    "confidence": "high",
    "location": "tests/test_config_deployment.py:269-275
(test_suggest_profile_no_cuda_probe_by_default)",
    "detail": "The spec/lens calls out the torch-free guarantee as critical, and
presets.py:14-16 + the loader comment at loader.py:169 (\"cheap, env-only (no torch)\") make it
a load-bearing claim: load_config() must never import torch. But NO test asserts this.
test_suggest_profile_no_cuda_probe_by_default only asserts suggest_profile({}) == \"\" (a
value), and the docstring 'no torch import' is never checked in code. Because torch IS installed
in this environment (verified: `python -c \"import torch\"` succeeds), a regression that made
the loader call suggest_profile(probe_cuda=True) ? importing torch on every config load ? would
still return '' for a no-GPU env and pass all 31 tests. I verified the gap directly: after
```

deleting torch from sys.modules and calling load_config('/nonexistent'), 'torch' is absent from sys.modules (current code is correct), but nothing pins it. The two torch tests (cuda_probe_false, cuda_available_never_raises) exercise the PROBE function, not the load path's torch-freeness.",
    "fix": "Add a test that pins the load path is torch-free: `def test_load_config_does_not_import_torch(monkeypatch): for m in [k for k in sys.modules if k=='torch' or k.startswith('torch.')]: monkeypatch.delitem(sys.modules, m, raising=False); load_config(NONEXISTENT); assert 'torch' not in sys.modules`. (Use monkeypatch.delitem so it restores torch after.) This is the one test that would FAIL if someone wired probe_cuda=True into the loader's suggest_profile() call or imported torch at module scope.",
    "lens": "tests"
  },
  {
    "title": "No-profile suggestion log-line (D15 'suggest, never apply') is untested ? the surfacing half of the no-surprise contract has no coverage",
    "severity": "minor",
    "confidence": "high",
    "location": "tests/test_config_deployment.py:220-229 (test_cloud_env_does_not_auto_apply_profile) vs loader.py:169-175",
    "detail": "The loader's no-profile branch calls suggest_profile() and, when the env suggests one, logs it at DEBUG (loader.py:171-175) so the operator sees 'environment suggests X ? set DEPLOYMENT_PROFILE to apply'. That log IS the D15 'never a surprise' surface in the manual-knobs path. test_cloud_env_does_not_auto_apply_profile sets K_SERVICE and asserts the config stays default (the 'never apply' half), but never asserts the suggestion was logged (the 'surface it' half). A regression that deleted the suggestion logging entirely ? silencing the only hint that the env wanted cloud ? would pass every test. Conversely there is no test that NO suggestion is logged when the env is bare (suggest_profile()==''), so a spurious-suggestion regression is also uncovered.",
    "fix": "Extend test_cloud_env_does_not_auto_apply_profile (or add a sibling) to capture caplog at DEBUG on logger 'backend.config.loader' and assert a record matching \"Environment suggests 'cloud-serving'\" exists ? and add a bare-env case asserting no such 'suggests' record is emitted. This pins both directions of the D15 suggestion surface.",
    "lens": "tests"
  },
  {
    "title": "compute_target consistency warning has only a positive (conflict?warn) test; no negative test that a matching compute_target stays silent",
    "severity": "nit",
    "confidence": "high",
    "location": "tests/test_config_deployment.py:162-170 (test_config_json_can_override_compute_target_with_warning) vs loader.py:156-163",
    "detail": "loader.py:158 warns only when config.json's compute_target differs from the profile's canonical (ct != canonical). test_config_json_can_override_compute_target_with_warning covers the conflict case (cpu-mock + cloud_run ? warn). But nothing asserts the warning is NOT emitted when the file sets compute_target to the canonical value (e.g. cpu-mock + cpu_mock), nor when the file omits compute_target. A regression that made the warning fire unconditionally (e.g. dropping the `ct != canonical` guard) would still pass ? operators would get a spurious 'overrides it' warning on every profile load and no test would catch it. This is a classic 'no negative-case' gap for a conditional log.",
    "fix": "Add a test: load config.json {deployment:{profile:'cpu-mock','compute_target':'cpu_mock'}} with caplog at WARNING on 'backend.config.loader' and assert NOT any('overrides it to' in r.message for r in caplog.records) ? i.e. a matching compute_target is silent.",
    "lens": "tests"
  },
  {
    "title": "Untested conflict: an invalid DEPLOYMENT_PROFILE env shadows a valid config.json profile, yielding an inconsistent loaded config (profile set, preset NOT applied)",
    "severity": "minor",
    "confidence": "high",
    "location": "loader.py:47-59 + 138-147 (no test in tests/test_config_deployment.py)",
    "detail": "Input: env DEPLOYMENT_PROFILE='banana', config.json {deployment:{profile:'cpu-mock'}}. _resolve_explicit_profile returns 'banana' (env wins, loader.py:51-53); is_known_profile fails so the loader warns and sets profile='' (loader.py:147) ? NO preset applied. But the file's raw deployment.profile='cpu-mock' survives in file_data and passes the

typed Literal, so the FINAL config is cfg.deployment.profile=='cpu-mock' while compute_target=='' and detector_impl=='auto' (preset un-applied). Verified live: profile='cpu-mock', compute_target='', detector_impl='auto'. So a recorded profile name with none of its preset effects ? a silently inconsistent state that contradicts the 'recorded profile == preset actually applied' invariant that test_env_profile_overrides_config_json_profile (line 183) pins for the all-valid case. No test exercises the bad-env + good-file combination.",
    "fix": "Add a test pinning the chosen contract for this conflict. Either (a) assert the documented-but-untested current behavior is intentional, or (preferable) make the loader fall back to the file's valid profile when the env value is unknown, then assert cfg.deployment.profile=='cpu-mock' AND cfg.deployment.compute_target=='cpu_mock' AND cfg.indexing.detector_impl=='stub' (preset applied) ? so a recorded profile always implies its preset.",
    "lens": "tests"
  },
  {
    "title": "env+file precedence test for non-profile knobs (env overrides a file value) is absent ? only the profile CHOICE precedence is tested",
    "severity": "nit",
    "confidence": "medium",
    "location": "tests/test_config_deployment.py:176-191 vs spec §4.3 precedence chain",
    "detail": "The spec's precedence is built-in defaults < profile preset < config.json < env < worker --set. The tests cover env-vs-file for the profile CHOICE (test_env_profile_overrides_config_json_profile) and file-vs-preset for ordinary knobs (test_config_json_overrides_preset_subkey). But there is no test that an ENV override of a non-profile knob (e.g. RALLY_DETECTOR_IMPL, named in loader.py:84 of presets and the spec) beats both the preset and config.json. This is partly because that env?knob wiring is applied 'at consumption sites downstream' (loader.py:131-132) and may be out of M1 scope ? hence medium confidence. If any such env override is in scope for M1, its top-of-precedence position is unpinned.",
    "fix": "Confirm whether non-profile env overrides (RALLY_DETECTOR_IMPL/WASB_*) are applied within load_config in M1. If yes, add a test that env beats both preset and config.json for that knob. If they are deferred to downstream consumption sites (out of M1 scope), add a one-line comment in the test module noting the env<--set tail of the precedence chain is intentionally tested elsewhere, so the omission is documented rather than accidental.",
    "lens": "tests"
  }
]

Adjudicate as a skeptical senior engineer. For each finding: independently re-check it against the actual code, DROP false positives and nits that don't matter for a pure/additive config PR, MERGE duplicates, and keep only genuinely actionable items. Return the final must-fix / should-fix list ordered by severity, each with a concrete fix. If the change is sound and ship-ready, say so plainly with empty findings.

2. assistant (2026-06-21T07:27:13.787Z)

I'll review M1 by reading the spec and all changed files, then verify each finding against the actual code.

3. user (2026-06-21T07:27:14.744Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      import logging
13      import os
14      from pathlib import Path
```

```
    from typing import Any

    from pydantic import BaseModel

    from .models import AppConfig
    from .presets import (
        COMPUTE_TARGET_FOR_PROFILE,
        PROFILE_PRESETS,
        is_known_profile,
        preset_for,
        suggest_profile,
    )

    logger = logging.getLogger(__name__)

    DEFAULT_CONFIG_PATH = Path("config.json")


    def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
        """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
scalars
        replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
        overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
        path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
        out = dict(base)
        for key, val in override.items():
            if isinstance(val, dict) and isinstance(out.get(key), dict):
                out[key] = _deep_merge(out[key], val)
            else:
                out[key] = val
        return out


    def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
        """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
none
        is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
        environment never silently selects a profile (and never silently spends on
cloud)."""
        env_profile = os.environ.get("DEPLOYMENT_PROFILE")
        if env_profile and env_profile.strip():
            return env_profile.strip()
        dep = file_data.get("deployment")
        if isinstance(dep, dict):
            prof = dep.get("profile")
            if isinstance(prof, str) and prof.strip():
                return prof.strip()
        return ""


    def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
        """Dotted paths in `data` that the typed config will not honor.

        Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
        kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
        typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
        user's intent is lost ? collect every such key so load_config can warn.
        """
        unknown: list[str] = []
        fields = model_cls.model_fields
        for key, val in data.items():
            if key not in fields:
```

```
                unknown.append(prefix + key)
                continue
            ann = fields[key].annotation
            if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
                unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
        return unknown


    def _get_builtin_defaults() -> dict[str, Any]:
        """Return a plain dict of the current built-in defaults.

        The AppConfig model is the single source of truth for defaults.
        """
        return AppConfig().model_dump(mode="json")


    def load_config(path: str | Path | None = None) -> AppConfig:
        """Load configuration with the following precedence:

        1. Built-in defaults defined in the Pydantic models.
        2. Values from the JSON file (if present and readable).
        3. (Future) environment / CLI overrides.

        Missing file or unreadable file ? returns a fully defaulted AppConfig.
        Unknown keys in the file are tolerated (extra="allow" on the model).
        """
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH

        file_data: dict[str, Any] = {}
        if cfg_path.exists():
            try:
                with open(cfg_path, "r", encoding="utf-8") as f:
                    file_data = json.load(f)
            except Exception as exc:
                # Non-fatal: continue with defaults + whatever we could parse.
                # In production we might log here.
                logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")

        # A config.json that is valid JSON but not an OBJECT (e.g. `[]`, `42`, `"x"`,
`null`) would
        # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
        # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
        # loader's "bad input is non-fatal" contract.
        if not isinstance(file_data, dict):
            logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
                           cfg_path, type(file_data).__name__)
            file_data = {}

        if file_data:
            unknown = _unknown_key_paths(file_data, AppConfig)
            if unknown:
                logger.warning(
                    "[CONFIG WARNING] %s contains keys the typed config does not "
                    "recognize (typo'd or removed knobs ? top-level extras are kept "
                    "but unread; unknown section keys are silently dropped): %s",
                    cfg_path, ", ".join(sorted(unknown)),
                )

        # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
```

```
        # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
        # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
        # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
        defaults = _get_builtin_defaults()
        profile = _resolve_explicit_profile(file_data)

        if profile and not is_known_profile(profile):
            # An unrecognised EXPLICIT profile is loud, not silent (the loader's "dropped
config"
            # ethos): warn and apply no preset. A bad value coming from config.json
additionally
            # hits the typed Literal at model_validate below (a hard, clear error); a bad
value
            # coming only from the env override is ignored here with this warning.
            logger.warning(
                "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
                profile, ", ".join(PROFILE_PRESETS),
            )
            profile = ""

        if profile:
            merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
            # Record the profile actually applied ? the env may have chosen it over
config.json,
            # so re-stamp it onto the merged deployment section to keep the record honest.
            merged.setdefault("deployment", {})
            if isinstance(merged["deployment"], dict):
                merged["deployment"]["profile"] = profile
                ct = merged["deployment"].get("compute_target")
                canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
                if ct and canonical and ct != canonical:
                    logger.warning(
                        "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
                        "config.json overrides it to %r ? honoring the explicit override.",
                        profile, canonical, ct,
                    )
                logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
                            profile, merged["deployment"].get("compute_target"))
        else:
            # No profile ? the original shallow merge (Pydantic re-defaults nested
sections).
            merged = {**defaults, **file_data}
            suggestion = suggest_profile()  # cheap, env-only (no torch); SUGGESTS only
(D15)
            if suggestion:
                logger.debug(
                    "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
                    "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
                    "auto-applies ? D15).", suggestion,
                )

        return AppConfig.model_validate(merged)


    def save_default_config(path: str | Path | None = None) -> Path:
        """Write a fresh default config.json.

        Used by scripts/doctor.py diagnostics and the --setup CLI path.
```

```
        Overwrites if the file exists.
        """
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
        cfg = AppConfig()
        cfg_path.parent.mkdir(parents=True, exist_ok=True)
        cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
        return cfg_path


    # Backwards-compatible alias used by some older call sites during transition.
    def get_default_config() -> dict[str, Any]:
        """Return defaults as a plain dict (for transitional use only)."""
        return _get_builtin_defaults()

```

4. user (2026-06-21T07:27:15.253Z)

```
      """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).

      A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
      config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
      knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).

      Precedence (lowest ? highest), realised by the loader::

          built-in model defaults  <  profile preset  <  config.json  <  env  <  worker --set

      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
      spend is never entered silently).
      """

      from __future__ import annotations

      import copy
      import os
      from typing import Any, Mapping, Optional

      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")

      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
          "truly-local": "local_gpu",
          "cloud-serving": "cloud_run",
          "cpu-mock": "cpu_mock",
      }

      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
      # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
```

```
extend
41      # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42      # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43      PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44          "truly-local": {
45              "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46              # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47              # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48              "indexing": {"skip_ai_handoff": True},
49              "storage": {"backend": "local"},
50          },
51          "cloud-serving": {
52              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53              # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54              "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55              "storage": {"backend": "gcs"},
56          },
57          "cpu-mock": {
58              "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59              # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60              "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
61              "storage": {"backend": "local"},
62          },
63      }
64
65
66      def is_known_profile(profile: str) -> bool:
67          """True for a non-empty, recognised profile name."""
68          return bool(profile) and profile in PROFILE_PRESETS
69
70
71      def preset_for(profile: str) -> dict[str, Any]:
72          """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
73          module-level table), or ``{}`` for ``""`` / an unknown profile."""
74          return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
75
76
77      def _truthy(val: Optional[str]) -> bool:
78          """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
79          if val is None:
80              return False
81          return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84      def _cuda_available() -> bool:
85          """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86          ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
87          try:
88              # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89              # and optional). Only reached when a caller opts into probe_cuda.
90              import torch
91
92              return bool(torch.cuda.is_available())
93          except Exception:
94              return False
95
96
97      def suggest_profile(
```

```
98          env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
99      ) -> str:
100         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
101
102         Signals, in priority order:
103
104           - ``K_SERVICE`` set (Cloud Run)        ? ``"cloud-serving"``
105           - ``CI`` truthy                        ? ``"cpu-mock"``
106           - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
107           - otherwise                            ? ``""`` (no strong suggestion)
108
109         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
110         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
111         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
112         """
113         e = os.environ if env is None else env
114         if e.get("K_SERVICE"):
115             return "cloud-serving"
116         if _truthy(e.get("CI")):
117             return "cpu-mock"
118         if probe_cuda and _cuda_available():
119             return "truly-local"
120         return ""
121
```

5. user (2026-06-21T07:27:15.496Z)

```
1       """Tests for the deployment-profile config (COMPUTE_DECOUPLED_SERVING M1).
2
3       What this pins:
4       - The default profile ("") applies NO preset ? the loaded config is byte-identical to
the
5         pre-M1 default, so the core is unaffected (the project's "default-OFF, zero core
change").
6       - Each profile applies its coherent preset bundle (compute_target + the existing
7         detector_impl / skip_ai_handoff / storage.backend knobs).
8       - Precedence: built-in defaults < profile preset < config.json (a file sub-key overrides
a
9         preset sub-key while sibling preset/default sub-keys survive ? i.e. real deep-merge
layering).
10      - D15: auto-detect SUGGESTS but never auto-applies (a bare cloud-like env does NOT
silently
11        select a profile / spend on cloud).
12      - The model's Literal enum stays in parity with backend/config/presets.PROFILE_PRESETS.
13      """
14
15      import json
16      import logging
17      import os
18
19      import pytest
20      from pydantic import ValidationError
21
22      from backend.config import AppConfig, DeploymentConfig, load_config, save_default_config
23      from backend.config.presets import (
24          COMPUTE_TARGET_FOR_PROFILE,
25          COMPUTE_TARGETS,
26          PROFILE_PRESETS,
27          PROFILES,
28          is_known_profile,
29          preset_for,
30          suggest_profile,
```

```
    )

    NONEXISTENT = "/nonexistent/path/12345/deployment_config.json"


    def _write(tmp_path, data) -> str:
        p = tmp_path / "cfg.json"
        p.write_text(json.dumps(data))
        return str(p)


    # ---------------------------------------------------------------------------
    # Defaults / additivity ? the default path must be unchanged.
    # ---------------------------------------------------------------------------
    def test_deployment_defaults_are_empty_and_inert():
        cfg = load_config(NONEXISTENT)
        assert isinstance(cfg.deployment, DeploymentConfig)
        assert cfg.deployment.profile == ""
        assert cfg.deployment.compute_target == ""


    def test_default_path_applies_no_preset(monkeypatch):
        """With no profile selected, the existing knobs keep their built-in defaults (no
preset
        mutates detector_impl / skip_ai_handoff / storage.backend)."""
        monkeypatch.delenv("DEPLOYMENT_PROFILE", raising=False)
        cfg = load_config(NONEXISTENT)
        assert cfg.indexing.detector_impl == "auto"
        assert cfg.indexing.skip_ai_handoff is False
        assert cfg.storage.backend == "local"


    def test_default_config_dump_only_adds_deployment_section():
        """Adding the section must not perturb any other default ? the dump equals a fresh
        AppConfig()'s dump, and the new section is present with empty values."""
        dumped = load_config(NONEXISTENT).model_dump(mode="json")
        assert dumped == AppConfig().model_dump(mode="json")
        assert dumped["deployment"] == {"profile": "", "compute_target": ""}


    # ---------------------------------------------------------------------------
    # Parity: the model Literal must match the presets table (no drift).
    # ---------------------------------------------------------------------------
    def test_profile_literal_matches_presets():
        import typing

        literal_profiles =
set(typing.get_args(DeploymentConfig.model_fields["profile"].annotation))
        assert literal_profiles == {"", *PROFILES}
        assert set(PROFILE_PRESETS) == set(PROFILES)
        assert set(COMPUTE_TARGET_FOR_PROFILE) == set(PROFILES)


    def test_compute_target_literal_matches_presets():
        import typing

        literal_targets =
set(typing.get_args(DeploymentConfig.model_fields["compute_target"].annotation))
        assert literal_targets == {"", *COMPUTE_TARGETS}
        assert set(COMPUTE_TARGET_FOR_PROFILE.values()) == set(COMPUTE_TARGETS)


    def test_each_preset_only_touches_existing_knobs():
        """A preset must only set knobs that exist on the typed models ? otherwise it would
be a
```

```
92          silent no-op. Validates the merged result against AppConfig for every profile."""
93          for profile in PROFILES:
94              # A preset's value set must validate cleanly when merged onto defaults.
95              merged = {**AppConfig().model_dump(mode="json")}
96              for section, kv in preset_for(profile).items():
97                  merged.setdefault(section, {})
98                  if isinstance(merged[section], dict):
99                      merged[section].update(kv)
100             AppConfig.model_validate(merged)  # raises if a preset key/value is invalid
101
102
103     # ------------------------------------------------------------------------------------
104     # Per-profile preset application.
105     # ------------------------------------------------------------------------------------
106     def test_cpu_mock_profile(tmp_path):
107         cfg = load_config(_write(tmp_path, {"deployment": {"profile": "cpu-mock"}}))
108         assert cfg.deployment.profile == "cpu-mock"
109         assert cfg.deployment.compute_target == "cpu_mock"
110         assert cfg.indexing.detector_impl == "stub"
111         assert cfg.indexing.skip_ai_handoff is True
112         assert cfg.storage.backend == "local"
113
114
115     def test_cloud_serving_profile(tmp_path):
116         cfg = load_config(_write(tmp_path, {"deployment": {"profile": "cloud-serving"}}))
117         assert cfg.deployment.profile == "cloud-serving"
118         assert cfg.deployment.compute_target == "cloud_run"
119         assert cfg.indexing.detector_impl == "native"
120         assert cfg.indexing.skip_ai_handoff is False  # Gemini-ON until the owned model
clears the floor
121         assert cfg.storage.backend == "gcs"
122
123
124     def test_truly_local_profile(tmp_path):
125         cfg = load_config(_write(tmp_path, {"deployment": {"profile": "truly-local"}}))
126         assert cfg.deployment.profile == "truly-local"
127         assert cfg.deployment.compute_target == "local_gpu"
128         # detector_impl is intentionally left at the production default (env picks WSL vs
native).
129         assert cfg.indexing.detector_impl == "auto"
130         assert cfg.indexing.skip_ai_handoff is True  # zero-cloud
131         assert cfg.storage.backend == "local"
132
133
134     # ------------------------------------------------------------------------------------
135     # Precedence: defaults < preset < config.json (deep-merge layering).
136     # ------------------------------------------------------------------------------------
137     def test_config_json_overrides_preset_subkey(tmp_path):
138         """A file sub-key beats the preset's sub-key, but a sibling preset sub-key in the
SAME
139         section survives (this is the deep-merge layering ? a shallow merge would wipe
it)."""
140         cfg = load_config(_write(tmp_path, {
141             "deployment": {"profile": "cpu-mock"},
142             "indexing": {"detector_impl": "native"},  # override one preset knob...
143         }))
144         assert cfg.indexing.detector_impl == "native"     # file wins
145         assert cfg.indexing.skip_ai_handoff is True       # ...preset sibling SURVIVES (deep
merge)
146
147
148     def test_preset_does_not_wipe_unrelated_defaults(tmp_path):
149         """Selecting a profile + setting an unrelated knob keeps the rest of the section at
its
150         built-in default (the preset and the file both layer onto the full defaults)."""
```

```
        cfg = load_config(_write(tmp_path, {
            "deployment": {"profile": "cpu-mock"},
            "indexing": {"ai_max_workers": 2},
        }))
        assert cfg.indexing.detector_impl == "stub"   # preset
        assert cfg.indexing.skip_ai_handoff is True   # preset
        assert cfg.indexing.ai_max_workers == 2       # file
        assert cfg.indexing.default_segmenter == "yolo_hybrid"  # untouched default
        assert cfg.indexing.max_window_duration == 6.0          # untouched default


    def test_config_json_can_override_compute_target_with_warning(tmp_path, caplog):
        cfg_path = _write(tmp_path, {
            "deployment": {"profile": "cpu-mock", "compute_target": "cloud_run"},
        })
        with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
            cfg = load_config(cfg_path)
        assert cfg.deployment.profile == "cpu-mock"
        assert cfg.deployment.compute_target == "cloud_run"  # explicit override honored
(D15)
        assert any("overrides it to 'cloud_run'" in r.message for r in caplog.records)


    # ---------------------------------------------------------------------------------
    # Env selection + precedence over config.json.
    # ---------------------------------------------------------------------------------
    def test_env_selects_profile(tmp_path, monkeypatch):
        monkeypatch.setenv("DEPLOYMENT_PROFILE", "cpu-mock")
        cfg = load_config(_write(tmp_path, {}))
        assert cfg.deployment.profile == "cpu-mock"
        assert cfg.indexing.detector_impl == "stub"


    def test_env_profile_overrides_config_json_profile(tmp_path, monkeypatch):
        """env wins over config.json for the profile CHOICE, and the recorded profile
matches the
        preset actually applied (not the file's value)."""
        monkeypatch.setenv("DEPLOYMENT_PROFILE", "cpu-mock")
        cfg = load_config(_write(tmp_path, {"deployment": {"profile": "cloud-serving"}}))
        assert cfg.deployment.profile == "cpu-mock"        # env value recorded
        assert cfg.deployment.compute_target == "cpu_mock"  # cpu-mock preset applied
        assert cfg.indexing.detector_impl == "stub"


    def test_blank_env_profile_is_ignored(tmp_path, monkeypatch):
        monkeypatch.setenv("DEPLOYMENT_PROFILE", "   ")
        cfg = load_config(_write(tmp_path, {}))
        assert cfg.deployment.profile == ""
        assert cfg.indexing.detector_impl == "auto"


    def test_unknown_env_profile_warns_and_applies_no_preset(tmp_path, monkeypatch, caplog):
        monkeypatch.setenv("DEPLOYMENT_PROFILE", "banana")
        with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
            cfg = load_config(_write(tmp_path, {}))
        assert cfg.deployment.profile == ""          # not applied
        assert cfg.indexing.detector_impl == "auto"  # no preset
        assert any("unrecognized deployment profile 'banana'" in r.message for r in
caplog.records)


    def test_unknown_profile_in_config_json_raises(tmp_path, monkeypatch):
        """A bad VALUE in config.json hits the typed Literal ? a loud, hard error (same
class as a
        bad `sport`), not a silent drop."""
```

```
        monkeypatch.delenv("DEPLOYMENT_PROFILE", raising=False)
        with pytest.raises(ValidationError):
            load_config(_write(tmp_path, {"deployment": {"profile": "banana"}}))


    # ------------------------------------------------------------------------------------
    # D15 ? auto-detect SUGGESTS but never AUTO-APPLIES.
    # ------------------------------------------------------------------------------------
    def test_cloud_env_does_not_auto_apply_profile(tmp_path, monkeypatch):
        """K_SERVICE (Cloud Run) makes suggest_profile() recommend cloud-serving, but with
no
        explicit profile the loaded config stays fully default ? NO silent cloud
selection."""
        monkeypatch.delenv("DEPLOYMENT_PROFILE", raising=False)
        monkeypatch.setenv("K_SERVICE", "khelsutra-serving")
        cfg = load_config(_write(tmp_path, {}))
        assert cfg.deployment.profile == ""           # NOT auto-applied
        assert cfg.deployment.compute_target == ""
        assert cfg.indexing.detector_impl == "auto"   # default, not "native"
        assert cfg.storage.backend == "local"         # default, not "gcs"


    def test_active_profile_is_logged(tmp_path, caplog):
        with caplog.at_level(logging.INFO, logger="backend.config.loader"):
            load_config(_write(tmp_path, {"deployment": {"profile": "truly-local"}}))
        assert any("deployment profile ACTIVE: truly-local" in r.message for r in
caplog.records)


    def test_no_active_profile_info_when_unset(tmp_path, caplog, monkeypatch):
        monkeypatch.delenv("DEPLOYMENT_PROFILE", raising=False)
        with caplog.at_level(logging.INFO, logger="backend.config.loader"):
            load_config(_write(tmp_path, {}))
        assert not any("profile ACTIVE" in r.message for r in caplog.records)


    # ------------------------------------------------------------------------------------
    # suggest_profile() ? pure env-only logic + the opt-in cuda probe.
    # ------------------------------------------------------------------------------------
    def test_suggest_profile_cloud_run():
        assert suggest_profile({"K_SERVICE": "svc"}) == "cloud-serving"


    def test_suggest_profile_ci():
        assert suggest_profile({"CI": "true"}) == "cpu-mock"
        assert suggest_profile({"CI": "1"}) == "cpu-mock"
        assert suggest_profile({"CI": "yes"}) == "cpu-mock"


    def test_suggest_profile_ci_falsy_or_absent():
        assert suggest_profile({}) == ""
        assert suggest_profile({"CI": "false"}) == ""
        assert suggest_profile({"CI": "0"}) == ""
        assert suggest_profile({"CI": ""}) == ""


    def test_suggest_profile_k_service_beats_ci():
        assert suggest_profile({"K_SERVICE": "svc", "CI": "true"}) == "cloud-serving"


    def test_suggest_profile_no_cuda_probe_by_default(monkeypatch):
        """probe_cuda defaults False ? no torch import, no truly-local suggestion even with
a GPU."""
        import backend.config.presets as presets

```

```
        monkeypatch.setattr(presets, "_cuda_available", lambda: True)
        assert suggest_profile({}) == ""                    # probe off ? no suggestion
        assert suggest_profile({}, probe_cuda=True) == "truly-local"


    def test_suggest_profile_cuda_probe_false(monkeypatch):
        import backend.config.presets as presets

        monkeypatch.setattr(presets, "_cuda_available", lambda: False)
        assert suggest_profile({}, probe_cuda=True) == ""


    def test_cuda_available_never_raises(monkeypatch):
        """Even a broken/missing torch must degrade to False, never propagate."""
        import builtins

        import backend.config.presets as presets

        real_import = builtins.__import__

        def boom(name, *a, **k):
            if name == "torch":
                raise RuntimeError("no torch")
            return real_import(name, *a, **k)

        monkeypatch.setattr(builtins, "__import__", boom)
        assert presets._cuda_available() is False


    # ---------------------------------------------------------------------------
    # Helpers + plumbing.
    # ---------------------------------------------------------------------------
    def test_is_known_profile():
        assert is_known_profile("cpu-mock") is True
        assert is_known_profile("") is False
        assert is_known_profile("banana") is False


    def test_preset_for_returns_copy():
        a = preset_for("cpu-mock")
        a["indexing"]["detector_impl"] = "mutated"
        b = preset_for("cpu-mock")
        assert b["indexing"]["detector_impl"] == "stub"  # module table not mutated
        assert preset_for("") == {}
        assert preset_for("banana") == {}


    def test_save_default_config_includes_deployment(tmp_path):
        p = save_default_config(str(tmp_path / "out.json"))
        saved = json.loads(p.read_text())
        assert saved["deployment"] == {"profile": "", "compute_target": ""}
        cfg = load_config(str(p))
        assert cfg.deployment.profile == ""


    def test_repo_config_json_has_no_profile():
        """The shipped config.json must not silently opt into a profile (default path stays
today's
        behaviour for the real deployment)."""
        repo_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
        cfg_path = os.path.join(repo_root, "config.json")
        if not os.path.exists(cfg_path):
            pytest.skip("no repo config.json")
        cfg = load_config(cfg_path)
        assert cfg.deployment.profile == ""
```

```
337
```

6. assistant (2026-06-21T07:27:18.036Z)

```
Now let me verify the models.py and the spec sections, plus run the actual triggering inputs to
confirm the findings.
```

7. user (2026-06-21T07:27:19.575Z)

```
310         single_folder_per_video: bool = False
311         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
312         workspace_root: str = ""
313         # Cloud namespace prefix under the root so per-video folders sit tidily beside
314         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
315         workspace_prefix: str = "workspaces"
316         local: dict[str, Any] = Field(default_factory=dict)
317         s3: dict[str, Any] = Field(default_factory=dict)
318         gcs: dict[str, Any] = Field(default_factory=dict)
319         gdrive: dict[str, Any] = Field(default_factory=dict)
320
321
322     class DeploymentConfig(BaseModel):
323         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
324         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
325
326         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
327         behaves exactly as before this section existed. A profile is opt-in (``config.json``
328         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
329         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
330         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
331         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
332
333         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
334
335         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
336         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
337         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
338         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
339         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
340
341
342     class AppConfig(BaseModel):
343         """Root configuration object.
344
345         All runtime configuration should be accessed via an instance of this class
346         (or the loader) rather than raw dict lookups.
347         """
348
349         sport: Sport = "badminton"
350         ai_provider: str = "gemini"
351         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
352         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
353         ai_model: str = "gemini-flash-latest"
354
355         validation: ValidationConfig = Field(default_factory=ValidationConfig)
```

```
356         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
357         output: OutputConfig = Field(default_factory=OutputConfig)
358         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
359         security: SecurityConfig = Field(default_factory=SecurityConfig)
360         logging: LoggingConfig = Field(default_factory=LoggingConfig)
361         storage: StorageConfig = Field(default_factory=StorageConfig)
362         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
363
364         # Allow extra keys during transition so existing config.json files do not break.
365         model_config = {
366             "extra": "allow",
367             "validate_assignment": True,
368         }
369
```

8. user (2026-06-21T07:27:19.755Z)

```
1       """Configuration package.
2
3       Public API:
4           from backend.config import load_config, AppConfig, Config
5
6       This package is the canonical place for all configuration access.
7       """
8
9       from .loader import load_config, save_default_config
10      from .models import (  # re-export common ones
11          AppConfig,
12          Config,
13          DeploymentConfig,
14          IndexingConfig,
15          StitchingConfig,
16          StorageConfig,
17          ValidationConfig,
18      )
19      from .presets import PROFILE_PRESETS, suggest_profile
20
21      __all__ = [
22          "load_config",
23          "save_default_config",
24          "AppConfig",
25          "Config",
26          "DeploymentConfig",
27          "IndexingConfig",
28          "StitchingConfig",
29          "StorageConfig",
30          "ValidationConfig",
31          "PROFILE_PRESETS",
32          "suggest_profile",
33      ]
34
```

9. assistant (2026-06-21T07:27:21.625Z)

Now let me verify the spec sections and run the actual triggering inputs to confirm the disputed findings.

10. user (2026-06-21T07:27:23.751Z)

```
1:# Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage
2-
3-> **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** · **Owner:**
Avi · **Created:** 2026-06-21 · **Last updated:** 2026-06-21
4-> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to
```

`docs/archives/past_projects/` when DONE)
5-> **Tracking anchors:** §7 progress tracker is the source of truth; pointer in memory `current-state` + `docs/NEXT_STEPS.md` once work starts.
6:> **Relation to existing docs:** the productionization successor to the archived `DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from [PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §3; the **truly-local** profile is the L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
7:> **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed **compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an afterthought.
8-> **Honesty note:** claims tagged `[verified]` (checked against code by the design workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
9-
10----
11-
12:> **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13, a headline feature), and execution is **never a surprise** ? auto-detect *suggests*, the user *confirms* (D15).
13-
14-## 0. TL;DR
15:The rally engine is a **pure function of `(input bytes + config) ? outputs`** in three deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile may branch on**. A **`ComputeBackend` (deployment profile)** decides *where* the core runs (a local process vs a Cloud Run GPU job); a **`StorageBackend`** decides *where bytes come from* (local FS / USB / mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local** (local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive / bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus **cpu-mock** for CI. The single most important caveat: detection is **byte-identical only on the same GPU** (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the **parity guarantee is enforced at the WINDOW + STITCH layer**, not detector-CSV bytes.
16-
17-## 1. Problem & goal
--
26-| # | Decision | Source | Implication |
27-|---|---|---|---|
28:| D1 | The core is a **pure 3-stage function** (DETECT/WINDOW/STITCH); **no profile branch inside it**. | design `[verified]` | All profile difference lives in config + the two registries. |
29:| D2 | **Two registries:** `ComputeBackend` (where) selected by `deployment.profile` + `compute_target`; `StorageBackend` (what-bytes) by `input_backend`/`storage.backend`. | ADR-014 | Realizes PLATFORM_ARCHITECTURE's `local\|hosted\|byo_worker`. |
30-| D3 | **Profiles shipped:** `truly-local`, `cloud-serving`, `cpu-mock` (CI); `byo_worker` reserved/deferred. | ADR-014 | Enum/seam reserved so BYO isn't precluded. |
31-| D4 | In **cloud-serving, STITCH runs on Cloud Run** (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger. | owner (a) | Stitch-on-laptop would hide real cost. |
--
38-| D11 | **Quality floor: Gemini handoff ON** for cloud-serving until the owned model clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a **Launch Report**; the owned model runs in **shadow/eval** meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |
39-| D12 | **Data-flywheel harness is IN SCOPE (owner add, Q5):** capture the (consented, adult) uploaded video + the Gemini reel/rally output ? **reliably store** in the per-video workspace/bucket ? **run shadow evals** (owned-vs-Gemini, dual-eval-set) ? **promote good ones to the golden TRAINING set** (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off). | owner 2026-06-21 | new **M11**; ties D10 (consent) + `data_pipeline/` + the eval/experiment harness. |
40:| D13 | **Profile is user-switchable ANYTIME ? a headline FEATURE.** The same user runs **local today, cloud tomorrow, back to local** ? it's just a config/startup choice; the pure core gives **equivalent** output either way. **Advertise it** ("your machine *or* our cloud ? your call, switch anytime"). | owner 2026-06-21 | Honest caveat: equivalent at the **rally/window** level, not bit-identical across GPUs (D5); a single job runs in one profile,

switching is *between* jobs. |
41:| D14 | **NORTHSTAR ? operate toward this:** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next to the source**. | owner 2026-06-21 | Implies a **`gdrive-api` (OAuth) StorageBackend** (read source + write reel back) ? distinct from the local **mount** backend; **resolves §8 #7**, new **M12**. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write); not necessarily v1, but the design must not preclude it (the `StorageBackend` ABC accommodates it). |
42:| D15 | **Auto-detect SUGGESTS, the user CONFIRMS ? execution is NEVER a surprise.** A GPU owner may still choose cloud; **cloud (= spend) is never entered silently**; the active profile is always explicit + surfaced. | owner 2026-06-21 | Amends §4.3 (auto-detect = a proposed default, not an auto-decision). |
43-
44-## 3. Foundation ? evolution / ADR lineage ? *trace the idea end-to-end*
--
48-|---|---|---|
49-| **ADR-005** | Permissive licensing (MIT/Apache-2.0/BSD only) | The engine + any served model + the **container image** (ffmpeg, fonts, weights) stay commercial-clean. |
50:| **ADR-008** | Owned-model topology; **train==serve featurizer single-sourcing** | The "core unaffected across profiles" is the same parity discipline, extended from train/serve to **local/cloud**. |
51:| **ADR-009** | KhelSutra Guru; **local-first delivery** | The **truly-local profile** is the direct continuation ? privacy/offline self-host stays first-class. |
52-| **ADR-010** | DECOUPLED: DO?GCP pivot | The cloud-compute origin (GPU + co-located bucket). |
53-| **ADR-011** | DECOUPLED scope = M0?M2+M3.5; **deferred M3 (serving) + M4 (billing/cost-guard)**; overrode the *"rent nothing / not a service"* posture | **This project realizes M3/M4.** Carries §06's owner-gated gates: **DPA/consent for non-owner uploads, collector `md5` keying, WASB-C3**. |
54-| **ADR-012** | GPU-native; TPU deferred; **Cloud Run+GPU scale-to-zero = serving model**; **NVDEC** named as the decode lever | The serving substrate + the 7-step seed scope this expands. |
55-| **ADR-013** | Drop DO; **GCP sole compute stack**; $200 DO credits ? non-compute only | The provider for cloud-serving; DO Spaces remains only a possible S3 *storage* target. |
56:| **? ADR-014 (new)** | **Compute-decoupled serving:** one pure core, profile-selected compute+storage; stitch-where-metered | This doc. **Refines** ADR-011/012, does **not** supersede them. |
57-| PLATFORM_ARCHITECTURE.md | The `ComputeBackend`/`StorageBackend` registries | The abstraction this realizes in code. |
58:| VIDEO_LOCALITY_MODEL.md / DESKTOP_APP_PLAN.md | L0 fully-local tier / `LocalDesktop` backend | The truly-local profile = these, productionized. |
59-| 07-COMPUTE-ABSTRACTION (archived) | `DeviceContext`/`DeviceProfile` (designed) | WASB hardcodes `device='cuda'` with **no CPU fallback** ? a portability gap M4 closes. |
60-| DEPLOY_REPORT_GCP_2026-06-20 (archived) | First real L4 run; **cross-GPU sub-pixel parity finding** | Why D5 (parity at window level, not CSV bytes). |
--
68-3. **STITCH (CPU/ffmpeg):** `VideoCompiler.compile_highlights(raw_video, segments, out_name) ? reel` (`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? ffprobe ? watermark ? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.
69-
70:**Non-goal (the trap to avoid):** *no* code branch inside detect/window/stitch keyed on profile ? the same discipline the experiment harness already enforces (a disabled cue is byte-identical to CONTROL).
71-
72:### 4.2 The deployment profiles
73-| Profile | Compute | Storage | Orchestration | When |
74-|---|---|---|---|---|
75-| **truly-local** | `compute_target=local_gpu`; `detector_impl=native` (Linux) / `auto` (WSL); stitch **in-process** | `local` (or `gdrive` = mounted Drive) | `python -m backend.main` or `worker infer` CLI; existing async job worker | Self-host / offline / privacy / dev-on-a-GPU-box ? owner (b) |
76-| **cloud-serving** | `compute_target=cloud_run`; `detector_impl=native` (L4, cuda); **detect AND stitch on the same Cloud Run job** | `gcs` (or gdrive mount / assembled upload); per-video workspace; reels via signed URL | Cloud Run control plane enqueues ? Cloud Run GPU job

pulls/runs/pushes; `WAIT_AND_RESUME` reschedules the control-plane job; scale-to-zero | Hosted SaaS ? owner (a) |
77:| **cpu-mock** | `compute_target=cpu_mock`; `detector_impl=stub` (synth/replay) | `local`/in-memory fakes | `run_all_tests.py` / pytest | CI, regression, profile-parity, GPU-free dev |
78-| *byo_worker* (deferred) | tenant GPU via outbound pull agent | tenant's own bucket (signed URLs) | long-poll pull; same job table scoped by tenant | Privacy-sensitive enterprise ? **DEFERRED**, enum/seam reserved |
79-

11. assistant (2026-06-21T07:27:28.084Z)

Now let me run the actual triggering inputs for the disputed findings to verify them empirically.

12. user (2026-06-21T07:27:29.337Z)

F1: profile='cloud-serving' compute_target=''

13. assistant (2026-06-21T07:27:31.485Z)

F1 confirmed: `profile='cloud-serving'` but `compute_target=''`. Now the env-typo findings (F2/F6 are duplicates).

14. user (2026-06-21T07:27:33.976Z)

[CONFIG] unrecognized deployment profile 'banana' (valid: truly-local, cloud-serving, cpu-mock); applying no preset.
F2/F6: profile='cpu-mock' compute_target='' detector_impl='auto'

15. assistant (2026-06-21T07:27:37.306Z)

F2/F6 confirmed (they're the same bug): env typo `banana` + file `cpu-mock` ? recorded `profile='cpu-mock'` but preset NOT applied (`detector_impl='auto'`, `compute_target=''`). Inconsistent state where the recorded profile lies about being active.

Let me verify the torch-free claim (F3) and run the test suite.

16. user (2026-06-21T07:27:39.292Z)

F3: torch imported by load_config? False

17. assistant (2026-06-21T07:27:41.708Z)

F3 confirmed: load path is torch-free (correct), but no test pins it. Let me run the full config test suite to confirm green baseline.

18. user (2026-06-21T07:27:49.940Z)

.... [100%]
52 passed in 5.57s

19. assistant (2026-06-21T07:28:01.132Z)

All 52 config tests pass. Now let me reason about adjudication.

Finding 1 (empty compute_target blanks active profile): Real and verified. But: compute_target is documented as inert in M1 (models.py:338-339 ? "consumed by the ComputeBackend seam from M2+"). Zero behavioral effect this milestone. It's an observability/honesty gap. The finding itself acknowledges "IMPACT IS BOUNDED." This is a should-fix-before-M2, minor.

Finding 2 = Finding 6 (env typo shadows valid file profile): Same bug, two reviewers. Verified. The recorded `profile='cpu-mock'` actively contradicts the applied behavior. This is the more material honesty bug ? a recorded profile name that asserts an active state with none

of its preset effects. The loader's own comment (loader.py:139-142) claims "a bad value coming only from the env override is ignored here" but it actually clobbers the file's valid profile. Genuine robustness gap, but only reachable under a specific misconfiguration (env typo co-existing with valid file profile). Minor, but the fix is clean (scrub the recorded profile to '' when no preset applied, or fall back to file profile).

Finding 3 (torch-free untested): Real test gap, the torch-free guarantee is load-bearing and a regression (wiring `probe_cuda=True` into the loader) would pass all tests since torch IS installed. High value, cheap test. Worth adding.

Findings 4, 5, 7 (test-coverage nits): Negative-case / log-surface assertions. These are genuine but low-priority for a pure/additive config PR. F5 and F7 are nits by the reviewers' own rating. F4 (D15 suggestion log untested) is minor but the suggestion log is DEBUG-only and advisory; lower priority.

Let me consolidate. The two substantive items are the env-typo inconsistency (F2/F6) and the torch-free test (F3). F1 is a worthwhile should-fix paired with the M2 boundary. The pure-test nits I'll fold down or drop.

20. user (2026-06-21T07:28:27.111Z)

Structured output provided successfully

21. assistant (2026-06-21T07:28:38.715Z)

Adjudication complete. I independently re-verified each finding by running the actual triggering inputs through `load_config` and the full config suite (52 tests pass ? the default `profile=''` path is byte-identical, so M1's pure/additive zero-core-change contract holds).

Kept (3 findings):

1. **Env-typo clobbers a valid file profile (minor, high)** ? merged the two duplicate reviewer findings (F2 + F6, same bug). Verified live: `DEPLOYMENT_PROFILE='banana'` + file `profile='cpu-mock'` records `profile='cpu-mock'` but applies no preset (`detector_impl='auto'`, `compute_target=''`). The recorded profile lies about being active, and the loader's own comment (loader.py:139-142) is wrong ? the env value doesn't get "ignored," it suppresses the file's valid profile. Reachable only under a misconfiguration, so minor, but the honesty-invariant violation is real and the fix is clean.

2. **Torch-free load path untested (minor, high)** ? the no-torch guarantee is load-bearing (presets.py:11-16, loader.py:169) but pinned only in docstrings. Verified the load path is genuinely torch-free today, yet a regression wiring `probe_cuda=True` into the loader would pass all tests because torch is installed. One cheap `monkeypatch.delitem` test closes it.

3. **Empty compute_target silently blanked (nit, high)** ? downgraded from the reviewer's "minor" to **nit** for M1 because compute_target is explicitly documented inert until M2 (models.py:338-339), so zero behavioral effect now. Flagged as must-revisit before M2 wires it to a real ComputeBackend.

Dropped:
- F4 (D15 suggestion DEBUG-log untested), F5 (negative compute_target-silent test), F7 (env-vs-knob precedence) ? genuine but low-value test-coverage nits for a pure/additive config PR; F7 also concerns env?knob wiring that loader.py:131-132 explicitly defers downstream (out of M1 scope). Not blocking, not worth the reviewer's ink here.

The PR is sound and ship-ready on its core contract. None of the three kept findings are correctness bugs in M1 behavior ? they are two honesty/observability gaps reachable only under misconfiguration plus one test-coverage gap. All file paths are absolute in the structured output.